

Moed A 2019 – Solution guidelines

August 9, 2019

Question 1

1. **The claim is correct.** Proof:

Consider the following two languages over $\Sigma = \{0, 1\}$:

$$L_1 = \{1^{2n} \mid n \in \mathbb{N}\}$$

$$L_2 = \{0^n 1^n \mid n \in \mathbb{N}\}$$

It is easy to verify that $\mathbf{ones}(L_1) = \mathbf{ones}(L_2) = \{1^{2n} \mid n \in \mathbb{N}\}$. In addition, $L_1 \in \text{REG}$ (for example, the regular expression $(11)^*$ describes it) while $L_2 \notin \text{REG}$ (seen in class).

2. **The claim is correct.** Proof:

We show that if $L \in \text{REG}$ then $\mathbf{ones}(L) \in \text{REG}$. Then, because REG is closed under concatenation we have that $L \cdot \mathbf{ones}(L) \in \text{REG}$.

If $L \in \text{REG}$, then there is a DFA $A = \langle Q, \Sigma, \delta, q_0, F \rangle$ with $L(A)$. We take A and convert it to an NFA $N = \langle Q, \{1\}, \delta', Q_0, F \rangle$ such that $L(N) = \mathbf{ones}(L)$. Intuitively, N is obtained from A by replacing all the letters in the transitions of A by the letter “1”. Formally, $\delta'(q, 1) = \{q' : \delta(q, \sigma) = q' \text{ for some } \sigma \in \Sigma\}$.

Let $w \in L$. Then, there is a sequence of states $q_0, \dots, q_{acc}$ that corresponds to the run of A on w . This sequence also corresponds to the run of N on $\mathbf{ones}(w)$, so $\mathbf{ones}(w) \in L(N)$.

In the other direction, if $w \in L(N)$, then there is a sequence of states $q_0, \dots, q_{acc}$ that corresponds to the run of N on w . Therefore, there exists $w' \in L$ such that $q_0, \dots, q_{acc}$ corresponds to the run of A on w' , and $\mathbf{ones}(w') = w$.

3. **The claim is correct.** Proof:

Consider the following languages:

$$L = \{x\#y\#z \mid x, y, z \in \{a, b\}^* \text{ are such that } x \text{ does not include } ab \text{ or } y = z\}$$

$$L' = \{x\#y\#z \mid x, y, z \in \{a, b\}^* \text{ and } x \text{ includes } ab\}$$

$$L'' = \{x\#y\#z \mid x, y, z \in \{a, b\}^*, x \text{ includes } ab \text{ and } y = z\}$$

Notice that $L'' = L \cap L'$. In addition, $L' \in \text{REG}$ and $L'' \notin \text{CFL}$ (proof below). However, if L was in CFL then from the closure of CFL to intersection with regular languages we get that $L'' \in \text{CFL}$, which is a contradiction.

Lemma: $L' \in \text{REG}$

Proof: The regular expression $(a+b)^*ab(a+b)^*\#(a+b)^*\#(a+b)^*$ describes it.

Lemma: $L'' \notin \text{CFL}$

Proof: We show that $L'' \notin \text{CFL}$ using the pumping lemma. Assume by way of contradiction that $L'' \in \text{CFL}$, and let p be its pumping constant. Let $w = ab\#a^p b^p \#a^p b^p$. Since $w \in L''$ and $|w| \geq p$, there exist words $u, v, x, y, z \in \Sigma^*$ such that $w = uvxyz$ and the three conditions of the pumping lemma are satisfied ($|vxy| \leq p$, $|vy| > 0$, $\forall i \in \mathbb{N}$, $uv^i xy^i z \in L''$).

We now consider five distinct cases:

- If vxy is contained in the first $a^p b^p$ (between the two $\#$ s), then let s be the number of a 's in vy and let t be the number of b 's in vy . According to the pumping lemma, $s + t > 0$. In addition, the word $uxz = ab\#a^{p-s} b^{p-t} \#a^p b^p \in L''$, which is of course not true.
- The case where vxy is contained in the second $a^p b^p$ (after the second $\#$) is symmetric to the previous case.
- If vxy is composed of parts from both halves of the first and second $a^p b^p$ and x contains $\#$, then because $|vxy| \leq p$ then vxy is contained in the last b^p of the first part and the first a^p of the second. Again, let s be the number of a 's in vy and let t be the number of b 's in vy . According to the pumping lemma, $s + t > 0$. In addition we must have that $vxy = ab\#a^p b^{p-t} \#a^{p-s} b^p \in L''$, but it is not true – contradiction.
- If vxy is contained in the part before the first $\#$, then because $|vxy| \leq p$, the word uxz is one of the following: $b\#a^p b^p \#a^p b^p$, $a\#a^p b^p \#a^p b^p$, $\#a^p b^p \#a^p b^p$. None of them is in L , which is in contradiction to the conditions of the pumping lemma.
- In any other case, we have that $\# \in v$ or $\# \in y$. Now, if we try to pump the word w it will result in an illegal word with incorrect number of $\#$ s. Again, contradiction.

Question 2

1. **The claim is incorrect.** Proof:

Let us assume in a way of contradiction that the claim is correct. We now show a mapping reduction from ALL_{TM} to $EMPTY_{TM}$. Given a machine $\langle M \rangle$, the reduction returns $f(\langle M \rangle)$. Because f is computable, the reduction is computable as well. In addition, the reduction is correct: Let $\langle M \rangle \in ALL_{TM}$. Therefore, $L(M) = \Sigma^*$ so $L(f(\langle M \rangle)) = \Sigma^* \setminus \Sigma^* = \emptyset$, so the reduction indeed returns a machine in $EMPTY_{TM}$. The other direction is similar.

Now, we know that $ALL_{TM} \in \overline{RE} \cap \overline{CoRE}$. However, $EMPTY_{TM} \in CoRE$, hence a contradiction.

2. **The language in $CoRE \setminus RE$.** Proof:

We start by showing that $L \in CoRE$. Given two machines $\langle M_1, M_2 \rangle$ and a number k , a machine M can verify that $\langle M_1, M_2, k \rangle \notin L$ in the following manner. M goes over every word in Σ^* in minlex order, and at step i simulates the run of M_1 and M_2 on the first i words for i steps. If for some step i both M_1 and M_2 accepted some word, M increases a counter by 1, and stops checking that word (so it will only count once). If at any step the counter is greater than k , M will reject $\langle M_1, M_2, k \rangle$.

And now for the interesting part. To show that $L \notin RE$ we start with a lemma.

Lemma: The language $ONE_{TM} = \{\langle M \rangle : |L(M)| \leq 1\}$ is not in RE .

Proof: We show a reduction from $EMPTY_{TM}$, which we know that is not in RE . The reduction works as follows. Given a machine $\langle M \rangle$ over some Σ , the reduction returns a new machine $\langle M' \rangle$ over $\Sigma \cup \{\#\}$, where $\# \notin \Sigma$. $L(M')$ is the union of the language of two machines – the first is M and the second is a machine that accepts a single word: $\#$. The reduction is computable, as constructing a machine that accepts a single word is computable, and constructing a machine that is the union of two other machines is also computable. The reduction is also correct: if $\langle M \rangle \in EMPTY_{TM}$, then $L(M') = \emptyset \cup \{\#\}$ and $|L(M')| \leq 1$, so $\langle M' \rangle \in ONE_{TM}$. The other direction is similar.

Back to the main proof. We show a reduction from ONE_{TM} to L . The reduction works as follows: Given an input word $\langle M \rangle$, the reduction returns $\langle M, M, 1 \rangle$. The reduction is correct, as if $\langle M \rangle \in ONE_{TM}$ then $|L(M) \cap L(M)| = |L(M)| \leq 1$. Otherwise, $|L(M) \cap L(M)| > 1$. In addition, the reduction is clearly computable.

Alternative reduction:

- Construction. Reduce $\overline{A_{TM}}$ to L : Given $\langle M, w \rangle$ the reduction outputs $\langle M', M', 1 \rangle$, where M' works as follows: Given x , M' simulates the run of M on w and answers the same.
- Correctness. If $\langle M, w \rangle \in \overline{A_{TM}}$ then M does not accept w . Therefore M' does not accept any $x \in \Sigma^*$. It means that $L(M') = \emptyset$ and $|L(M') \cap L(M')| = |\emptyset| = 0$, and thus $\langle M', M', 1 \rangle \in L$. If $\langle M, w \rangle \notin \overline{A_{TM}}$ then M accept w . So M' accepts every $x \in \Sigma^*$. Therefore $L(M') = \Sigma^*$ and thus $|L(M') \cap L(M')| = |\Sigma^*|$ which means that $\langle M', M', 1 \rangle \notin L$.
- Computability. We have seen that simulating the run of a given TM on a word is computable, so the reduction is computable.

Question 3

1. **The claim is incorrect.** Proof:

We look at the languages $L_1 = \{0\}$ and $L_2 = \{1\}$ over $\Sigma = \{0, 1\}$. Notice that $L_1 \neq L_2$ and both languages are in NP (they are in P and $P \not\subseteq NP$). In addition, $L_1 \leq_p L_2$. Indeed, given a word $w \in \Sigma^*$, the reduction checks if $w = 0$, and if so the reduction returns 1, else 00. This reduction is obviously correct and polynomial, because it takes linear time, hence polynomial time, to check that $w = 0$. Similarly, we can show that $L_2 \leq_p L_1$.

According to the claim this means that both L_1 and L_2 are NPC. Since they are also in P this means $P = NP$, contradicting the assumption $P \neq NP$.

2. **The claim is correct.** Proof: We first show that $L \in NL$. The NTM works as follows:

```

v ← s;
c ← 0;
while c < 2 · |V| do
    guess a neighbour u of v;
    v ← u;
    c ← c + 1;
    if v = t and c is even then
        | accept;
    end
end
reject;
```

Algorithm 1: NTM for the language L

This machine only need to save the current vertex and t , which can be done in logarithmic space. We now show that the machine indeed decides L .

Lemma: If there is a path of even length from s to t in G , then there is a path of even length from s to t in G of length smaller than $2|V|$.

Proof: let $s = v_0, v_1, \dots, v_m = t$ a path of even length from s to t of minimal length (among the paths of even length). We name this path 'P'. Assume by a way of contradiction that $m \geq 2|V|$. Then, there are more than $2|v|$ vertexes in P. from the pigeonhole principle there are three indexes $i < j < k$ such that $v_i = v_j = v_k$. If the path from v_i to v_j is of even length then the path $s = v_0, v_1, \dots, v_i, v_{j+1}, \dots, v_m = t$ is also of even length because we removed a path of even length from P (which is itself of even length). This, is contradiction to the minimalism assumption of P. Similarly, if the path from v_j to v_k is of even length then $s = v_0, v_1, \dots, v_j, v_{k+1}, \dots, v_m = t$ is of even length. Finally, and if both the path from v_i to v_j and v_j to v_k are odd then

$s = v_0, v_1, \dots, v_i, v_{k+1}, \dots, v_m = t$ is of even length (both paths are removed). All in contradiction to the minimalism assumption of P . Therefore, $m \leq 2|v|$.

Now, if $\langle G, s, t \rangle \in L$ then from the lemma there is a path P of even length from s to t which is smaller than $2|v|$. Therefore, there exists an accepting run of M that guesses all the vertexes in P . In the other direction, if M accepts, then the series of guessed vertexes defines a path of even length from s to t . Therefore, M decides L .

To show that L is NL-hard, we will show a reduction from $PATH$. The reduction works as follows. Given a word $\langle G, t, s \rangle$ where $G = \langle V, E \rangle$, the reduction returns $\langle G', t, s \rangle$ where G' is the graph G with a self loop in s (if one wasn't already exist). Obviously, the reduction can be done in logarithmic space, as all we did is adding a single edge to G . The reduction is also correct. Indeed, if $\langle G, t, s \rangle \in PATH$ then there is a path from s to t in G . This path also exists in G' . However if this path is not of even length, we can just use the self loop in s to make it even, so $\langle G', t, s \rangle \in L$. In the other direction, if $\langle G, t, s \rangle \notin PATH$ then there is no path between s and t in G' , as adding a self loop will not connect s and t if they weren't already connected in G .

Alternative reduction from $PATH$: Instead of adding a self loop, the reduction constructs a new graph G' in which each edge is being “doubled”. That is, for each edge $(u, v) \in E$ create edges $(u, w), (w, v)$ where w is a new vertex. That way, we double the length of any path from s to t , so if there is a path between the vertexes in G we have a path of even length in G' . The details of the proof are omitted.

3. **The language is NP-complete.** Proof: We next show that $SHORTHAM \in NP$ and $SHORTHAM \in NP\text{-hard}$, therefore $SHORTHAM \in NPC$. We first show that $SHORTHAM \in NP$. An NTM for $SHORTHAM$ works like the NTM for D-s-t-HAMPATH, with only one difference – after the machine correctly guesses and verifies a hamilton path from s to t , it also counts the number of edges in the path, and accepts only if it is at most $\frac{|E|}{2}$. The additional step also takes polynomial time. Therefore $SHORTHAM \in NP$.

To show that $SHORTHAM$ is NP-hard, we show a reduction from D-s-t-HAMPATH, which we proved to be NP-hard. The reduction works as follows. Given a word $\langle G, t, s \rangle$ where $G = \langle V, E \rangle$, the reduction returns $\langle G', t, s \rangle$ where G' is the graph G with edges from t to all other vertexes.

This reduction is polynomial in the number of vertexes because for each one we added at most a single edge. We next prove the correctness of the reduction. If $\langle G, t, s \rangle \in D\text{-s-t-HAMPATH}$ then there is a hamilton path from s to t in G . Its length is exactly $|V| - 1$, otherwise we would have encountered the same vertex twice. The same path exists in G' , but now G' contains at least $2|V| - 2$ edges ($|V| - 1$ original edges constructing the path and new $|V| - 1$ edges from t). Therefore that path is of length at most $\frac{|E|}{2}$. Hence $\langle G', s, t \rangle \in SHORTHAM$. for the other direction, note that if $\langle G', s, t \rangle \in SHORTHAM$, then G' has a hamilton path from s to t . None of the path's edges are from t . Therefore the same path exists in G , hence $\langle G, s, t \rangle \in D\text{-s-t-HAMPATH}$.